

Building a Real-Data RICD Adapter

An Instructional Manual, Drawn from a Real Build

Every lesson here was learned by making the mistake first, on real data, and catching it. This is written for whoever builds the next one — a different domain, a different adapter — so they don't have to make the same mistake to learn the same thing.

1. Data Verification Discipline

1.1 Check the Dictionary Every Time, Not Once

A government or institutional data source's variable codes do not mean the same thing across years by default. A finance form used one reporting standard for one range of years and a genuinely different one afterward; treating a code as stable across time without checking its dictionary entry for that specific year produced a real, silent error that looked like a modeling problem until the actual cause was found.

- Before trusting any variable code, open that year's own data dictionary and confirm what it actually measures — do not assume continuity from a prior year's confirmed meaning.
- Watch for institution-type or reporting-standard mismatches: a form built for one category of institution (public, private, for-profit, two-year) may not apply, or may apply under a different code, to another.
- A plausible-sounding label is not the same as the correct total. Two named sub-categories that both 'sound like' spending are not the same as genuine total expenditure — check what a quantity is actually the sum of before using it as an accumulator input.

1.2 When Something Looks Broken, Suspect the Label Before the Model

Across this project, the single most common failure pattern was: a result looks alarming or nonsensical, and the first instinct is to suspect the underlying theory or tool is broken. In every real instance this session, the actual cause was mislabeled or misidentified data — a finance form assumed to be one type when it was another, a request for data that was already in hand under a different name, an institution's UNITID capturing the wrong organizational sub-unit entirely.

- Default assumption: the tool's basic architecture is sound, and one or two isolated bugs are fixable and specific, not evidence the whole approach is wrong.
- Default first move when a result looks wrong: verify the data's identity and labeling before doubting the mechanism that produced the result.
- This extends to institutional identity itself — a tracked ID may capture only a sub-unit of a much larger, differently-performing enterprise. Verify what entity a real-world label actually refers to before trusting a real-world grade or reputation applied to it.

2. Model Architecture — Common Traps

2.1 A Parameter With No Likelihood Is Not a Real Parameter

A threshold or classifier variable can be given a prior, sampled, and reported with apparent confidence — while never actually being connected to any real outcome through a likelihood term. Its posterior is then just its own prior, regardless of how confident it looks. This produced a genuine, damning result in this project: a parameter meant to

signal collapse risk reported a stable, thriving institution as a confirmed collapse risk for over a decade, because nothing in the model had ever told it what a real collapse or a real non-collapse actually looked like.

- Before trusting any threshold or classification output, confirm it is tied to a real observed outcome through an actual likelihood, not merely sampled from a plausible-looking prior.
- A parameter that 'sounds meaningful' in the model's narrative description is not evidence it is doing real work computationally.

2.2 Non-Centered Parameterization for Random Walks

A centered random-walk parameterization, where the step size is itself a fitted parameter, is a well-known sampling trap (a 'funnel' geometry that standard samplers struggle with). Use a non-centered form — fixed-scale raw steps multiplied by a separately-fit scale parameter — as the default, not an optimization to reach for only after sampling problems appear.

2.3 An Incomplete Proxy Can Structurally Prevent Its Own Purpose

Building an accumulator meant to detect financial distress from only a partial slice of real expense categories, rather than genuine total expenditure, can leave that accumulator structurally unable to ever activate — regardless of how severe the real underlying problem is. This is not a sensitivity issue to tune; it is a wiring error that makes the intended signal impossible to produce at all. Confirm an accumulator's real inputs sum to the actual total the theory calls for, not a convenient or partial substitute.

3. Classifying What Kind of Process an Observable Actually Is

Before modeling any real observable as a latent process, ask what kind of process actually generated it — this determines which tool is correct, not merely which tool is convenient.

3.1 Drift-Type vs. Shock-Type Observables

A quantity that evolves through many small, roughly homogeneous increments (a slowly-shifting staffing composition, gradual enrollment change) is drift-type, and an ordinary continuous-noise latent process is the right tool. A quantity that evolves through long stable stretches punctuated by rare, large, discrete jumps (a sudden debt escalation, an abrupt regulatory action) is shock-type, and forcing it through the same continuous-drift tool will systematically understate its real severity — not because of a bad prior, but because a homogeneous-step model has no way to represent 'mostly small steps, rarely a large one' at all.

- Diagnostic: does the real history of this quantity look like gradual accumulation, or like stability interrupted by rare large events? The latter needs heavy-tailed or explicit jump structure, not a standard Gaussian random walk.
- A heavy-tailed (e.g., Student-t) latent process is a practical, sampler-friendly way to represent rare jumps without a discrete jump-indicator variable, at the honest cost of harder sampling geometry — a real and acceptable trade-off for correctly representing the process, not a sign of an unstable model.
- Confirmed directly in this project: modeling a genuinely shock-type quantity as drift-type understated a real, known severity value by nearly an order of magnitude. The fix, not the failure, was the informative event.

3.2 Some Diagnostics Need Raw Data, Not Model Output

A diagnostic built to detect a sharp, synchronized shock across many independent units failed twice when run on Bayesian model-derived latent quantities, and worked cleanly the first time it was run on raw, unprocessed data instead. A model's own shared-prior structure smooths exactly the kind of sharp signal such a diagnostic is meant to

find. Any diagnostic explicitly designed around detecting sudden, discrete events should run upstream of a model's own fitting, on the rawest available adapter-supplied observable, not on that model's smoothed output.

3.3 A Trend Measure Inherits the Blind Spots of Whatever It Was Built From

A trend feature derived from one channel of real data (in this project, enrollment and completion) will not automatically see acceleration happening in a different, related channel (resource debt) — the two can be driven by genuinely different underlying processes that do not move in lockstep, especially around a sudden event like a debt default. A single, persistently-misclassified case that shows extreme severity on one channel alongside a merely mild trend on a different channel is a real signal that the existing trend measure may be structurally blind to what is actually happening, not evidence the case is simply an unresolvable exception. Building a second, channel-specific trend measure directly from the channel showing the severity — rather than continuing to rely on a trend inherited from an unrelated channel — is worth testing directly before concluding a case cannot be resolved.

4. Feature and Label Validation

4.1 Audit Labels With the Same Rigor as Features

The single largest improvement in this project's classifier accuracy came not from a new feature, but from correcting labeling errors using evidence that had already been gathered earlier in the same project and never connected back to the training data. A model that keeps misclassifying the same few cases should prompt an audit of whether those cases are correctly labeled at all, not only an attempt to add a feature that might fix them.

- If several persistent 'hard cases' cluster around units that were never independently verified, verify them before trusting the model's disagreement with their labels as a model failure.
- A confirmed-clean label and a merely-assumed-clean label are not the same thing, even if both currently read the same way in a dataset.

4.2 A Well-Motivated Feature Can Still Make Things Worse

Multiple theoretically sound candidate features, each with a real, defensible mechanism behind them, were tested honestly via held-out validation and made overall performance worse, not better. With a limited real sample size, adding a feature costs real overfitting risk that a plausible-sounding mechanism does not automatically offset. Test every candidate feature against held-out cases before trusting it, regardless of how sound its underlying theory sounds — and if it fails, growing the real sample is usually the right next lever, not further feature engineering on the same small set.

4.3 A Failed Single Feature May Still Work as an Interaction

A feature that shows zero discriminating value on its own can still carry real signal once combined with a second, related feature. A raw enrollment-share metric intended to detect a specific type of institutional decline failed completely as a standalone predictor, because the same raw number is produced by two genuinely different underlying processes (a healthy, resourced expansion and a cost-driven, hollow one) that a single number cannot distinguish. Multiplying it against a second signal that separates those two processes (a shift toward cheaper, less-supported staffing) recovered real, validated predictive value the standalone feature never had. Before discarding a theoretically well-motivated feature that fails alone, consider whether it is actually one half of a real interaction rather than a complete, standalone signal.

4.4 Distinguishing a Genuine Signal Conflict From Small-Sample Noise

When a single case persistently misclassifies despite an otherwise-strong model, it is worth asking directly whether that persistence reflects a real, stable disagreement or an artifact of too little data to draw a confident boundary —

these look identical on a single leave-one-out result but call for opposite responses, and a genuine, direct test can tell them apart rather than leaving the question to intuition.

Bootstrap-resample the training data many times (holding the case in question out each time), refit, and record how often the model lands on the wrong answer. A result near a coin flip across resamples is the signature of noise: the boundary simply isn't pinned down yet, and adding more real, similar examples to the training set should be expected to resolve it, which can itself be tested as a falsifiable prediction — in one real instance, adding real institutions in batches moved a case's wrong-answer rate from over ninety percent down through the noise threshold in a clean, monotonic trend, exactly as the noise diagnosis predicted. A result that stays robustly wrong regardless of which other cases are used for training — in another real instance, wrongly predicted the same way in ninety-five percent of resamples — is a genuinely different finding: a real, stable disagreement the current features cannot resolve, which more data of the same kind is unlikely to fix. The second kind of result should be reported as an honest, open exception, not iterated on indefinitely in the hope that one more feature or one more institution will make it go away — sometimes it will, and testing a specific, theoretically-motivated fix is worthwhile, but the case should be allowed to remain unresolved, reported plainly as such, if it does not.

5. Denominators and Normalization

5.1 A Raw Count Confounded With Scale Is Not Yet a Feature

A count-based metric pulled from real data (in this project, research retraction counts) can correlate almost perfectly with an organization's raw scale rather than with whatever underlying quality or integrity question it was meant to answer. Before treating any raw count as a usable signal, check whether it is simply re-measuring size under a different name. A genuine rate — the count divided by a real, comparable denominator (in this case, total research output) — is what actually answers the intended question; the raw count alone does not.

- If every small unit in a dataset shows near-zero on a count-based metric and every large unit shows a count proportional to its own scale, the metric is very likely confounded with size and is not yet ready to use as a standalone feature.

5.2 Normalize by a Genuine, Independent Scale Measure — Not by a Quantity Correlated With Scale

A stock quantity (in this project, institutional financial reserves) needs to be compared against something to be meaningful across units of very different size, and the choice of denominator is not interchangeable. Normalizing a reserve measure by a flow or liability quantity (debt) can look like a real fix on a limited, homogeneous subset of units, then fail once units spanning a much wider range of absolute scale are included — because a larger unit's debt tends to scale up roughly in proportion to its overall size too, so dividing reserves by debt only partially removes the size variation the normalization was meant to control for, and the part left behind is exactly what breaks the result once truly large units enter the comparison.

- Prefer a genuinely independent scale measure (population served, physical footprint, headcount — something that does not itself move with the quantity being normalized) over a correlated flow or liability quantity, even when the correlated quantity is more readily at hand.
- A normalization that works on a smaller, more homogeneous subset should be re-tested specifically against the most extreme, largest-scale units available before being trusted on the full population — this project's own version of this mistake looked like a complete success until public research universities, several orders of magnitude larger than the rest of the panel, were added back in.

5.3 Causal Normalization Needs a Stated, Specific Rule

"Normalize only using information available up to the current time" is necessary but not sufficient — it does not by itself specify whether the normalization range expands forever, uses a fixed trailing window, or is fixed once during an initial calibration period. These produce genuinely different trajectories from the same raw data. State explicitly

which construction is being used, since a new extreme value can otherwise silently change the meaning of every previously-reported normalized value under a rolling or expanding scheme, in a way that looks like a real change in the underlying system but is actually just the coordinate system shifting underneath it.

6. Multi-Unit Coupling

When representing a relationship between two real, separate entities (a merger, a formal transfer of obligations, an extraction or dependency relationship), prefer a real, currently-identifiable, documented case over an invented or hypothetical coupling structure wherever one exists. A genuine, quantified real-world event — even one not yet fully realized in observed data — gives a concrete, falsifiable basis for the coupling parameters that an abstract, unanchored coupling weight cannot provide. Where the real event postdates the available data window, state the resulting output plainly as a forward projection rather than a retrospective validation, and identify what future data would actually confirm or disconfirm it.

7. The General Discipline Underneath All of This

Nearly every real advance in this project came from the same basic pattern: build the most obvious, straightforward version of something; test it honestly against real, independently-verified outcomes rather than assuming it works; and when it fails, treat the specific shape of the failure as the actual answer to what needs to be built instead, rather than as an obstacle to route around or a reason to abandon the approach.

A negative result, properly diagnosed, is not a wasted effort. In this project, a failed feature revealed the real interaction that worked; a failed model-integration attempt revealed the correct classification of the underlying process; a failed diagnostic on the wrong kind of data revealed exactly which kind of data it actually needed. None of these fixes would have been found by reasoning about the problem in the abstract — they were found by building the wrong thing first, carefully, and reading its failure precisely.